



**POLYTECHNIQUE
MONTRÉAL**

UNIVERSITÉ
D'INGÉNIERIE

INF3500 - Conception et réalisation de systèmes numériques

Laboratoire 1

Marc-Antoine MANNINGHAM
Beaurel FOHOM

Automne 2025

Vérification de code-barres

À la fin de ce laboratoire, vous devrez être capable de :

- concevoir un module combinatoire pour qu'il rencontre ses spécifications
- concevoir un banc d'essai pour vérifier qu'un module combinatoire rencontre ses spécifications
 - utiliser des fonctions en VHDL
 - utiliser des types abstraits
 - utiliser l'énoncé *assert*
- faire la synthèse et l'implémentation du module, et programmer un FPGA pour en vérifier le fonctionnement

Ce laboratoire s'appuie sur et complète le matériel suivant :

1. Les concepts couverts dans le laboratoire 0
2. La matière du cours des thèmes 2 (Technologies de logique programmable) et 3 (Modélisation et vérification de circuits combinatoires).

! Évaluation

Les boîtes comme celle-ci indiquent les points qui seront évalués dans votre dépôt Git et votre rapport. Prêtez-y attention pour obtenir le maximum de points !

Liste des fichiers

Fichier	Contenu
src/code_barre.vhd	Module de détection de validité de code-barres
src/code_barre_tb.vhd	Banc d'essai du module <code>code_barre</code> , qui lui envoie des stimuli pour valider son bon fonctionnement en simulation
src/top.vhd	<i>Top level</i> du projet
xdc/PYNQ-Z2-v1.0.xdc	Fichier de contraintes pour la PYNQ-Z2
synth-impl/create_zynq_wrapper.tcl synth-impl/zynq.tcl	Scripts de génération du <i>wrapper</i> pour le Zynq, à sourcer après avoir importé les sources

Partie 1 : Conception d'un module combinatoire

Une compagnie développe une imprimante à code-barres à haute vitesse. Elle réalise cependant que certaines impressions ont des défauts. On vous demande alors de concevoir un système pour inspecter les impressions.

Un code-barres est constitué d'une suite de bandes blanches 0 et noires 1 en alternance. Une bande blanche peut mesurer au moins 1 pixel de large. Les bandes noires mesurent exclusivement 1 pixel de large.

Exemple de codes valides :

0	1	0	0	0	1	0	1
---	---	---	---	---	---	---	---

0	0	0	0	1	0	0	0
---	---	---	---	---	---	---	---

Vous devez développer un module qui va vérifier la validité des codes imprimés. Ce module aura plusieurs sorties :

- `o_erreur_encre_vide` : aucune bande n'est détectée
- `o_erreur_impression` : si une bande noire ne mesure pas exactement 1 pixel de large
- `o_impression_valide` : si aucune erreur n'est détectée dans le code

En fonction du résultat, vous devrez aussi utiliser la première led RGB pour indiquer un code couleur :

- Rouge : Erreur d'impression
- Bleu : Encre vide
- Vert : Impression valide

Votre architecture doit fonctionner pour toute largeur de code N.

! Évaluation

Complétez le fichier `code_barre.vhd` et remettez-le. **N'apportez aucun changement aux noms des entités, aux noms des architectures ni à la liste des ports ni à leurs noms.**

Utilisez l'ébauche de banc d'essai fournie dans le fichier `code_barre_tb.vhd` pour stimuler votre module.

Partie 2 : Conception d'un banc d'essai

Complétez et remettez la description de banc d'essai fourni dans le fichier `code_barre_tb.vhd` pour qu'il vérifie le comportement de votre module selon les spécifications données dans la partie 1. Pour minimiser les chances d'erreur, vous devez implémenter

une logique différente de celle utilisée dans votre module pour calculer les sorties attendues dans le banc d'essai, et comparer ces sorties attendues avec les sorties du module pour chaque valeur possible de l'entrée.

Vérifiez le fonctionnement correct de votre code de la partie 1 avec votre banc d'essai. Au besoin, corrigez votre code de la partie 1.

! Évaluation

Complétez le fichier `code_barre_tb.vhd` et faites un *push* de vos modifications. Ajoutez une capture d'écran de simulation à votre rapport.

Partie 3 : Synthèse et implémentation sur la carte

Utilisez la même procédure qu'au laboratoire 0 pour configurer votre projet Vivado.

Pour cette partie, on fixe la largeur du code `N` à 32.

Vérifiez le fonctionnement correct de votre système sur la carte. Utilisez l'interface PYNQ avec la même procédure qu'au laboratoire 0.

! Évaluation

Remettez votre fichier de configuration `top.bit` lorsque tout fonctionne. Ajoutez aussi à votre dépôt le *notebook* Jupyter que vous avez utilisé pour valider votre design.

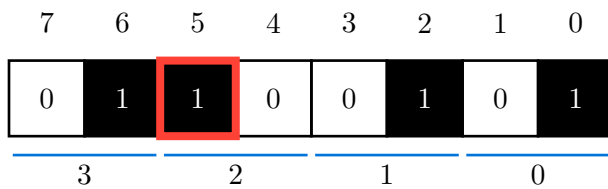
Partie 4 : Défi - au-delà du A

Mise en garde. *Compléter correctement les parties précédentes peut donner une note de 17 / 20 (85%), ce qui peut normalement être interprété comme un A. La partie 4 demande du travail supplémentaire qui sort normalement des attentes du cours. Il n'est pas nécessaire de la compléter pour réussir le cours ni pour obtenir une bonne note. Il n'est pas recommandé de s'y attaquer si vous éprouvez des difficultés dans un autre cours. La partie 4 propose un défi supplémentaire pour les personnes qui souhaitent s'investir davantage dans le cours INF3500 en toute connaissance de cause.*

4a. Afficher la position de la première erreur

Le système doit pouvoir donner la position de la première erreur. La position de l'erreur doit être placée sur `o_erreur_position`. Celle-ci sera visible avec l'interface PYNQ. Regardez le fichier `top.vhd` pour comprendre comment le signal est connecté.

Par exemple, le code suivant va donner 5 :



La zone d'impression est divisée en 4 secteurs. Un indicateur affiche le premier secteur où se trouve une erreur sur le port `o_erreur_secteur`. Par exemple, si la première erreur se trouve dans le secteur 0, le bit 0 du port `o_erreur_secteur` doit être à '1' et les autres à 0 ; si l'erreur est dans le secteur 3, le bit numéro 3 de `o_erreur_secteur` doit être à '1' et les autres à '0'. S'il n'y a aucune erreur, tous les bits de `o_erreur_secteur` sont à '0'.

Votre architecture doit fonctionner pour toute largeur de code `N`.

! Évaluation

Modifiez vos fichiers `code_barre.vhd` et `code_barre_tb.vhd` pour implémenter et vérifier cette nouvelle fonctionnalité.

4b. Étude de l'utilisation des ressources de votre module selon le nombre d'entrées

Faites plusieurs synthèses (et non l'implémentation au complet) de votre module `code_barre` uniquement (et non du module `top`) pour différentes largeurs. Obtenez un rapport des ressources utilisées par la synthèse pour plusieurs valeurs de `N`. Vous pouvez utiliser une boucle en TCL (`help for`) pour automatiser le processus.

i Info

Pour synthétiser uniquement le module `code_barre` en attribuant la valeur `<n>` à `N`, utilisez la commande

```
synth_design -top code_barre -generic N=<n> -part xc7z020clg400-1 -assert
```

Puis, obtenez le rapport d'utilisation des ressources :

```
report_utilization [-file monrapport.txt [-append]]
```

- L'option `-file` spécifie un fichier dans lequel Vivado doit écrire son rapport d'utilisation. En son absence, le rapport sera écrit dans la console TCL.
- Si l'option `-append` est passée avec `-file`, le rapport est ajouté à la fin de `monrapport.txt`. Sinon, `monrapport.txt` est réécrit avec le nouveau rapport.

! Évaluation

Étudiez la progression de la complexité du circuit quand la largeur du détecteur varie. Présentez vos résultats dans un tableau de la même forme que celui ci-dessous et un graphique pour le nombre de LUTs. Faites une analyse et présentez une discussion dans votre `rapport.pdf`.

N	Slice LUTs	Slice Registers	F7 Muxes	F8 Muxes	Bonded IOB
8					
16					
24					
32					
48					
64					

Tableau 1. – Tableau à intégrer (complété) à votre rapport

Remise

La remise se fait directement sur votre dépôt Git. Faites un *push* régulier de vos modifications, et faites un *push* final avant la date limite de la remise. Respectez l'arborescence de fichiers originale. Consultez le barème de correction pour la liste des fichiers à remettre.

Directives spéciales :

- Ne modifiez pas les noms des fichiers, entités ou architectures, ni les listes de `generics` ou de ports
- Remettez du code de très bonne qualité, lisible, bien indenté et commenté.
- Indiquez clairement la source de tout code que vous réutilisez ou dont vous vous inspirez.
- Remettez votre rapport dans le fichier `rapport.pdf` à la racine du dépôt

Barème de correction

Le barème de correction est progressif. Il est relativement facile d'obtenir une note de passage (> 10) au laboratoire et il faut mettre du travail pour obtenir l'équivalent d'un A (17/20). Obtenir une note plus élevée (jusqu'à 20/20) nécessite plus de travail que ce qui est normalement demandé dans le cadre du cours et plus que les 9 heures que vous devez normalement passer par semaine sur ce cours.

Critères	Points
Partie 1 Votre module final dans le fichier <code>code_barre.vhd</code>	6
Partie 2 - Votre banc d'essai complet dans le fichier <code>code_barre_tb.vhd</code>, avec une logique différente de celle du module pour le calcul des sorties et des vérifications à l'aide de <code>assert</code> - Votre capture de simulation dans votre rapport	7
Partie 3 - Votre <i>bitstream</i> <code>top.bit</code> - Votre <i>notebook</i> Jupyter	2
Général Qualité, lisibilité et élégance du code : indentation, choix des identificateurs, qualité et pertinence des commentaires respect des consignes de remise incluant les noms des fichiers, orthographe, etc.	2
Sous-total	17
Partie 4 - Vos fichiers <code>code_barre.vhd</code> et <code>code_barre_tb.vhd</code> modifiés (4a.) - Vos données sous forme tabulaire et graphique dans votre fichier <code>rapport.pdf</code>, une analyse et une discussion (4b.)	2 1
Total	20